

Module 17 – Generative AI Fundamentals

1. Introduction to Generative AI

What is Generative AI?

Generative AI (GenAI) is a type of Artificial Intelligence that creates new content such as text, code, images, audio, videos, and documents based on user prompts.

Difference Between Generative AI and Traditional AI

Generative AI	Traditional AI
Creates new content	Analyzes existing data
Generates text, images, code	Predicts or classifies data
Example: ChatGPT, GitHub Copilot	Example: Spam Filter, Face Recognition

Applications of Generative AI

- Text Generation
- Code Generation
- Chatbots
- Image Generation
- Video Generation
- Content Creation
- Software Development

History of Generative AI

Year	Development
1960s	Early Chatbots (ELIZA)
2014	GANs (Generative Adversarial Networks)
2020	GPT-3
2022	ChatGPT
Present	GitHub Copilot, GPT-4, Gemini, Claude

2. Prompt Engineering

What is Prompt Engineering?

Prompt Engineering is the process of writing effective instructions (prompts) to obtain accurate and useful responses from AI models.

Prompting Techniques

Zero-shot Prompting

The model performs a task without any examples.

Example:

```
Write a Java program to find the factorial of a number.
```

Few-shot Prompting

The model is provided with one or more examples before performing the task.

Example:

```
Input: 5  
Output: 120
```

```
Input: 6  
Output: 720
```

Chain-of-Thought Prompting

The model is instructed to solve the problem step by step.

Example:

```
Explain step by step how Binary Search works.
```

Best Practices for Prompt Engineering

- Write clear instructions.
- Provide sufficient context.
- Specify the expected output format.
- Break complex tasks into smaller prompts.
- Refine prompts based on responses.

Ethical Considerations

- Avoid biased prompts.
- Verify AI-generated information.
- Protect sensitive data.
- Use AI responsibly.
- Respect privacy and copyright.

3. Introduction to GitHub Copilot

What is GitHub Copilot?

GitHub Copilot is an AI-powered coding assistant developed by **GitHub and OpenAI**. It helps developers write code faster by providing intelligent code suggestions.

Features of GitHub Copilot

- Code completion.
- Function generation.
- Boilerplate code generation.
- Documentation generation.
- Test case generation.
- Code refactoring.
- Bug fixing suggestions.

How GitHub Copilot Works

GitHub Copilot analyzes:

- Current code
- Comments
- Function names
- File context

It then suggests relevant code using AI models.

Supported IDEs

- Visual Studio Code
- Visual Studio
- JetBrains IDEs
- Neovim

Supported Languages

- Java
- Python
- JavaScript
- TypeScript
- C#
- C++
- Go
- PHP
- Ruby
- Kotlin

4. Setup and Configuration

Installing GitHub Copilot

1. Open Visual Studio Code.
2. Open Extensions.
3. Search **GitHub Copilot**.
4. Click **Install**.
5. Sign in with your GitHub account.
6. Enable Copilot.

First Coding Task

Example Prompt:

Write a Java program to check whether a number is prime.

Copilot automatically suggests the implementation.

5. Core Features and Capabilities

Code Suggestions

Automatically suggests code while typing.

Function Generation

Generate complete functions from comments.

Example:

```
// Write a method to reverse a string
```

Documentation Generation

Automatically generates comments and documentation.

Example:

```
/**  
 * Calculates the factorial of a number.  
 */
```

Code Refactoring

Improves existing code by making it cleaner, shorter, and more efficient.

Test Case Generation

Generates unit test cases automatically.

Example:

- JUnit Test Cases
- NUnit Test Cases
- PyTest Cases

6. Security and Ethical Considerations

Risks of AI-Generated Code

- Security vulnerabilities.
- Incorrect code (hallucinations).
- Outdated coding practices.
- Performance issues.

Licensing and Attribution

AI-generated code may resemble existing open-source code. Developers should verify licensing requirements before using it in projects.

Data Privacy

Avoid sharing:

- Passwords.
- API Keys.
- Personal Information.
- Confidential company code.

Only share necessary information with AI tools.

Responsible Use of GitHub Copilot

- Review all generated code before using it.
- Test the generated code thoroughly.
- Verify correctness and security.
- Do not rely completely on AI.
- Follow coding standards and best practices.

7. Advantages of Generative AI

- Increases developer productivity.
- Automates repetitive coding tasks.
- Generates documentation.
- Speeds up software development.
- Assists in debugging.
- Helps beginners learn programming.

8. Limitations of Generative AI

- May generate incorrect code.
- Can produce biased responses.
- Requires human verification.
- Privacy concerns.
- May not understand complete project context.

9. Best Practices

- Write clear and specific prompts.
- Provide sufficient context.
- Verify AI-generated responses.
- Protect confidential information.
- Test generated code before deployment.
- Use AI as an assistant, not as a replacement for developers.

10. Conclusion

Generative AI is transforming software development by enabling developers to generate code, documentation, and other content quickly. Prompt Engineering helps users obtain accurate AI responses, while GitHub Copilot acts as an intelligent coding assistant that improves productivity. Responsible use, proper validation, and awareness of security and ethical considerations are essential for effectively using AI-powered development tools.